Bayesian networks

Baye Malick Gning

Table des matières

1 Bayesian networks as modeling tool **3**

2 Conditional independence **5**

3 Free parameters **5**

4 Inference **7**

5 Conditional independence in Bayesian network and its rules **8**

6 Bayesian networks and causality **10**

7 Learning from complete data : MLE of Bayesian network **10**

8 Bayesian inference in Bayesian networks **11**

9 Naive Bayes **12**
9.1 Model fitting 13
9.2 Bayesian Naive Bayes 16

10 Calibration of output distributions **17**

11 Classification-related notions **17**
11.1 False positive vs false negative tradeoff 17
11.2 Precision recall curves 19
11.3 F-score 19

12 References **20**

1 Bayesian networks as modeling tool

Definition 1.1 (Graph)**.** A graph G consists of vertices (nodes) and edges (links) between the vertices. Edges may be directed or undirected. A graph with all edges directed is called a **directed graph**, whereas a graph with all edges undirected is called an **undirected graph**.

Definition 1.2 (Path, ancestors, descendants)**.** A **path** is a sequence of vertices that connects two nodes. The vertices A such that $A \to B$ and $B \not\to A$ are the **ancestors** of B. The vertices B such that $1 \to B$ and $B \not\to A$ are the **descendants** of A

Definition 1.3. DAG A **directed acyclic graph** (DAG) is a graph G with directed edges on each link between the vertex, such that by following a path from one node to another along the direction of each edge, no vertex will be revisited. The figure 1.1 illustrates a DAG and figure 1.2 show what is not a DAG.

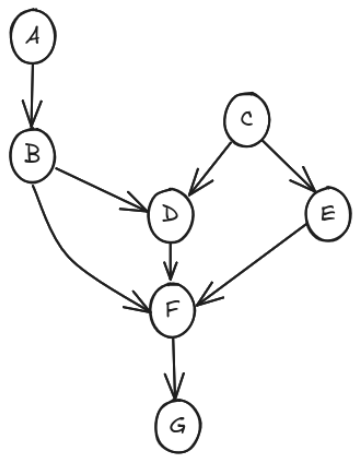


FIGURE 1.1 – This is a DAG

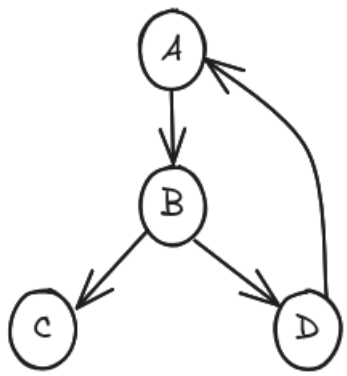


FIGURE 1.2 – This is not a DAG

Definition 1.4. Bayesian network A DAG whose vertices $\mathbf{X} = \{X_i | 1 \leq i \leq n\}$ are random variables is a Bayesian network if and only if the joint distribution over $\mathbf{X}$ is of the form :

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i|\text{parents}(x_i)) \tag{1.1}$$

Not every model necessarily has a Bayesian Network (BN) representation, but many probabilistic models can be structured as a BN if they satisfy certain conditions. The fewer arcs, the more independent relationships.

Example 1.1. Joint distribution of DAG from figure 1.1

$$p_{A,\ldots,G}(a, b, \ldots, g) = p_A(a) \times p_{B|A=a}(b) \times p_C(c) \tag{1.2}$$
$$\times\, p_{D|B=b,C=c}(d) \times p_{E|C=c}(e) \tag{1.3}$$
$$\times\, p_{F|B=b,D=d,E=e}(e) \times p_{G|F=f}(g) \tag{1.4}$$

Example 1.2 (Wet Grass)**.** One morning, Teresie leaves her house and realizes that her grass is wet. Is it due to overnight rain, or did she forget to turn off the sprinkler last night ? Next, she notices that the grass of her neighbor, Jack, is also wet. This explains away, to some extent, the possibility that her sprinkler was left on. and she concludes, therefore, that it has probably been raining.

We model the above situation using probability by following the general modeling graph approach. First, we define the variables we wish to include in our model :
$R \in \{0, 1\}$, where $R = 1$ means that it has been raining and 0 otherwise.
$S \in \{0, 1\}$, where $S = 1$ means that Teresie has forgotten to turn off the sprinkler and 0 otherwise.
$J \in \{0, 1\}$, where $J = 1$ means that Jack's grass is wet and 0 otherwise.
$T \in \{0, 1\}$, where $T = 1$ means that Teresie's grass is wet and 0 otherwise.
Our variable of interest, the joint probability, is $P(T, J, R, S)$ (the order of variables is irrelevant) :

$$P(T, J, R, S) = P(T|J, R, S)P(J, R, S) \tag{1.5}$$
$$= P(T|J, R, S)P(J|R, S)P(R, S) \tag{1.6}$$
$$= P(T|J, R, S)P(J|R, S)P(R|S)P(S) \tag{1.7}$$

Since these variables in this example can take one of two states, it would appear that we naively have to specify the values for each of the $\mathbf{2^4 = 16}$ **states**. However, since there are normalization conditions for probabilities, we do not need to specify all the states' probabilities.

The first term $P(T|J, R, S)$ requires specifying $2^3 = 8$ values. We need $P(T = 1|J, R, S)$ for the 8 joint states of J, R, S, and the value $P(T = 0|J, R, S)$ is given by normalization, i.e., $P(T = 0|J, R, S) = 1 - P(T = 1|J, R, S)$

Similarly, we need :
$P(J|R, S) \quad \rightarrow \quad$ 4 states
$P(R|S) \quad \rightarrow \quad$ 2 states
$P(S) \quad \rightarrow \quad$ 1 states

This gives a total of 15 values. In general, for the distribution of n binary variables, we need to specify $2^n - 1$ values in the range $[0, 1]$. One drawback of this approach is that the number of values that need to be specified scales exponentially, making it impractical for large n.

2 Conditional independence

We often know constraints of the system. This leads to making **conditional independence** assumptions.

Example 2.1. For example, in the example of wet grass(1.2), we may assume that Tracy's grass being wet depends only directly on whether or not it has been raining and whether or not the sprinkler was on. That is, we write : $P(T|J, R, S) = P(T|R, S)$.
Since whether or not Jack's grass is wet is influenced only directly by whether or not it has been raining, we write : $P(J|R, S) = P(J|R)$
Since the rain is not directly influenced by the sprinkler, we also have : $P(R|S) = P(R)$
This means our model now simplifies to :

$$P(T, J, R, S) = P(T|R, S)P(J|R)P(R)P(S)$$

We can represent this condition graphically as in Figure 2.1.

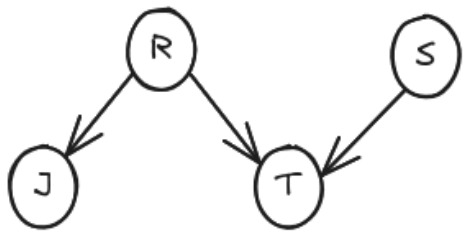


FIGURE 2.1 – Bayesian network for wet grass example 1.2

This reduces the number of values that we need to specify to : $\mathbf{4 + 2 + 1 + 1 = 8}$ a significant saving over the 15 values required when no conditional independence had been assumed.

To complete the model, we need to numerically specify the values of each **conditional probability table** (CPT). The prior probabilities for R and S will be given by $p(R = 1)$, $p(S = 1)$, $p(J = 1|R = 1)$, $p(J = 0|R = 0)$, etc.

3 Free parameters

Number and cardinality of parents mostly determine the model complexity, e.g. in discrete case :

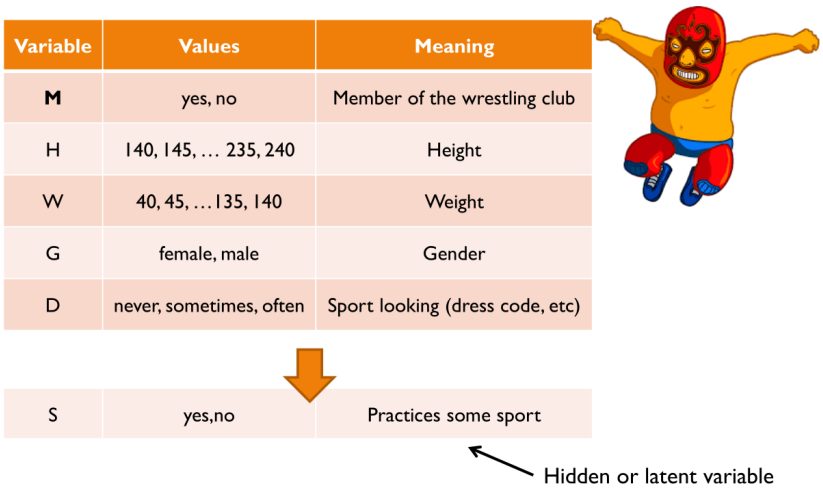
$$\text{number of free params} = \sum_{i=1}^{n} (|V_i| - 1) \prod_{V \in \text{parent}(V_i)} |V| \tag{3.1}$$

Variable name	Number of values	Number of parent value combinations	Size of CPT
G	2	1	$1 \times 1 = 1$
H	20	2	$19 \times 2 = 38$
M	2	2×20	$1 \times 40 = 40$
W	20	$2 \times 2 \times 20 = 80$	$19 \times 80 = 1520$
D	3	$2 \times 2 \times 20 = 80$	$2 \times 80 = 160$
Total			1759

TABLE 3.1 – Free parameters table of wrestling club problem from figure 3.1

Example 3.1. Let's compute number of free parameters from the wrestling club example whom information are given in figure 3.1. The DAG is given in figure 3.2. Thus, we derive the number of free parameters as follows :
This leads to **1759** values instead of $2400 = 1 \times 20 \times 2 \times 20 \times 3$. But it is not so good because when there is insufficient data to estimate all the parameters in a Bayesian network, it can lead to unreliable conditional probability table (CPT) estimates. With too many parameters and sparse data, many combinations of parent values may not appear in the dataset, causing missing entries in the CPTs and increasing the risk of overfitting. This makes the model less reliable and reduces its ability to generalize to new data.
One solution is to use continuous random variables and assume parameterized distributions. For example, $H|G = g \sim \mathcal{N}(\mu_g, \sigma_g^2)$. This involves 4 parameters : $\mu_m, \sigma_m^2, \mu_f, \sigma_f^2$ (m for male and f for female). For continuous parents, introduce parameterized dependency with them, e.g. $W|G = g, H = h, M = m \sim \mathcal{N}(\mu_{g,m}^0 + \mu_{g,m}^0 h, \sigma_{g,m}^{2^0} + \sigma_{g,m}^{2^0} h)$. This involves 16 parameters : $\mu_{g,m}^0, \sigma_{g,m}^{2^0}$ for $g \in \{m, f\}$ and $m \in \{y, n\}$ (where y stands for yes and n stands for no).

Thus we reduced the number of parameters and prevent from overfitting : $4 + 16$ instead of $38 + 1520$ parameters



Variable	Values	Meaning
M	yes, no	Member of the wrestling club
H	140, 145, … 235, 240	Height
W	40, 45, …135, 140	Weight
G	female, male	Gender
D	never, sometimes, often	Sport looking (dress code, etc)
S	yes,no	Practices some sport

Hidden or latent variable

FIGURE 3.1 – Information about wrestling club example

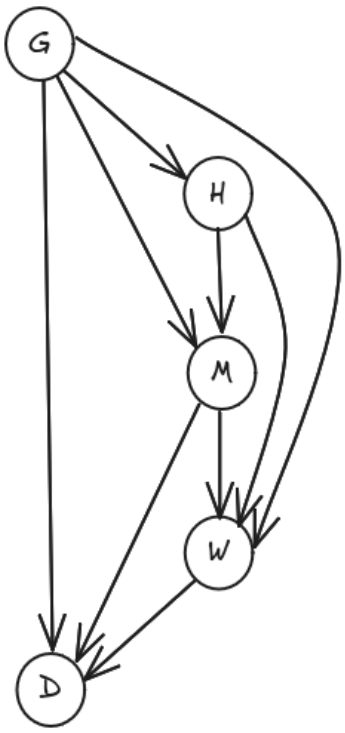


FIGURE 3.2 – BN for wrestling club example

4 Inference

Now that we've can define a model, we can perform inference.

Example 4.1. Let's come back to wet grass example (1.2) and derive the probability that the sprinkler was on overnight, given that Tracey's grass is wet : $p(S = 1|T = 1)$. To do this, we use Bayes' rule :

$$p(S = 1|T = 1) = \frac{p(S = 1, T = 1)}{p(T = 1)} \tag{4.1}$$

$$= \frac{\sum_{J,R} p(T = 1, J, R, S = 1)}{\sum_{J,R,S} p(T = 1, J, R, S)} \tag{4.2}$$

$$= \frac{\sum_{J,R} p(J|R)p(T = 1|R, S = 1)p(R)p(S = 1)}{\sum_{J,R,S} p(J|R)p(T = 1|R, S)p(R)p(S)} \tag{4.3}$$

$$= \frac{\sum_{R} p(T = 1|R, S = 1)p(R)p(S = 1)}{\sum_{R,S} p(T = 1|R, S)p(R)p(S)} \tag{4.4}$$

Let us now derive the probability that Tracey's sprinkler was on overnight, given that her grass is wet and that Jack's grass is also wet, $p(S = 1|T = 1, J = 1)$. We use Bayes' rule again :

$$p(S=1|T=1,J=1) = \frac{p(S=1,T=1,J=1)}{p(T=1,J=1)} \tag{4.5}$$

$$= \frac{\sum_R p(T=1,J=1,R,S=1)}{\sum_{R,S} p(T=1,J=1,R,S)} \tag{4.6}$$

$$= \frac{\sum_R p(J=1|R)p(T=1|R,S=1)p(R)p(S=1)}{\sum_{R,S} p(J=1|R)p(T=1|R,S)p(R)p(S)} \tag{4.7}$$

5 Conditional independence in Bayesian network and its rules

A Bayesian network (BN) corresponds to a set of conditional independence assumptions. It is not always immediately clear from the DAG whether a set of variables is conditionally independent of a set of other variables.

Definition 5.1. Conditional independence Let $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$ be sets of random variables. We say that $\mathcal{X}$ is conditionally independent of $\mathcal{Y}$ given $\mathcal{Z}$, denoted as : $\mathcal{X} \perp \mathcal{Y}|\mathcal{Z}$ if and only if the joint probability distribution factorizes as :

$$P(\mathcal{X},\mathcal{Y}|\mathcal{Z}) = p(\mathcal{X}|\mathcal{Z})p(\mathcal{Y}|\mathcal{Z}), \tag{5.1}$$

for all sets of variables $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$.
If $\mathcal{X}$ and $\mathcal{Y}$ are not conditionally independent, they are conditionally dependent, denoted as : $\mathcal{X} \not\perp \mathcal{Y}|\mathcal{Z}$.

Definition 5.2 (Collider)**.** In a Bayesian network, a collider is a situation where two variables, say X and Y, both have a common descendant, say Z. We illustrate it in the figure 5.1. In graphs (a) and (c), Z is not a collider and represent conditionnal indepencance $X \perp Y|Z$. In (b), Z is a collider. In graphs (b) and (d), X and Y are conditionally dependant given variable Z ($X \not\perp Y|Z$).

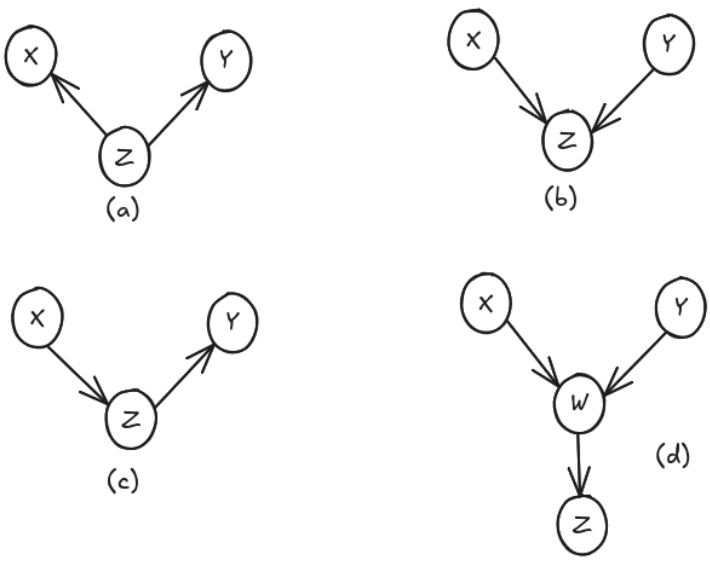


FIGURE 5.1 – Collider and rules

In the figure 5.2, we the variable Z is **not observed** and we wonder if X and Y are independent or not according the graph. The figure 5.2 gives the rules for independence in BN in case of **not observed** variable.

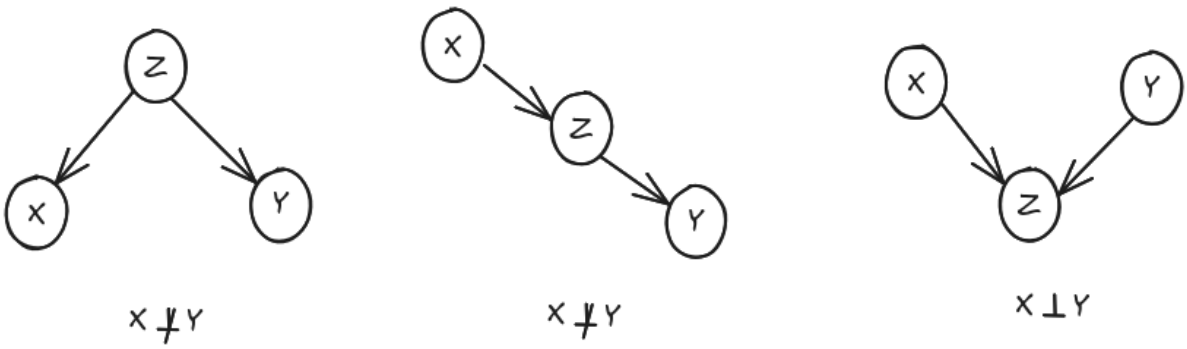


FIGURE 5.2 – Rules of independence when Z is **not observed**

In the figure 5.3, we the variable Z is **observed** and we illustrate new rules for independence in BN in case of **observed** variable.

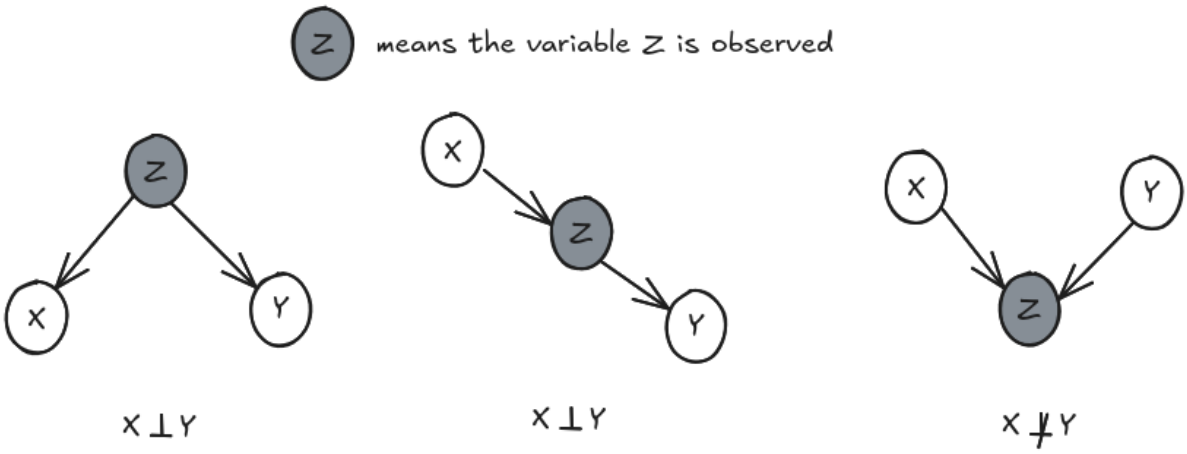


FIGURE 5.3 – Rules of independence when Z is **observed**

In the context of BN, the concepts of d-separation and d-connection are central to determining conditional independence with structure given by the DAG.

Definition 5.3 (d-connection, d-separation)**.** If G is a directed graph in which $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$ are disjoint sets of vertices, then $\mathcal{X}$ and $\mathcal{Y}$ are d-connected by $\mathcal{Z}$ in G if and only if there exists an undirected path U between some vertex in $\mathcal{X}$ and some vertex in $\mathcal{Y}$ such that for every collider C on U, either C or a descendant of C is in $\mathcal{Z}$, and no non-collider of U is in $\mathcal{Z}$.
$\mathcal{X}$ and $\mathcal{Y}$ are d-separated by $\mathcal{Z}$ in G if and only if they are not d-connected by $\mathcal{Z}$ in G.
One may also phrase this as follows : for every variable $x \in \mathcal{X}$ and $y \in \mathcal{Y}$, check every path U between x and y. A path U is said to be **blocked** if there is a node w on U such that either :

1. w is a collider and neither w nor any of its descendants is in $\mathcal{Z}$, or
2. w is not a collider on U and w is in $\mathcal{Z}$.

We represent in figure 5.4 three blocking configuration.

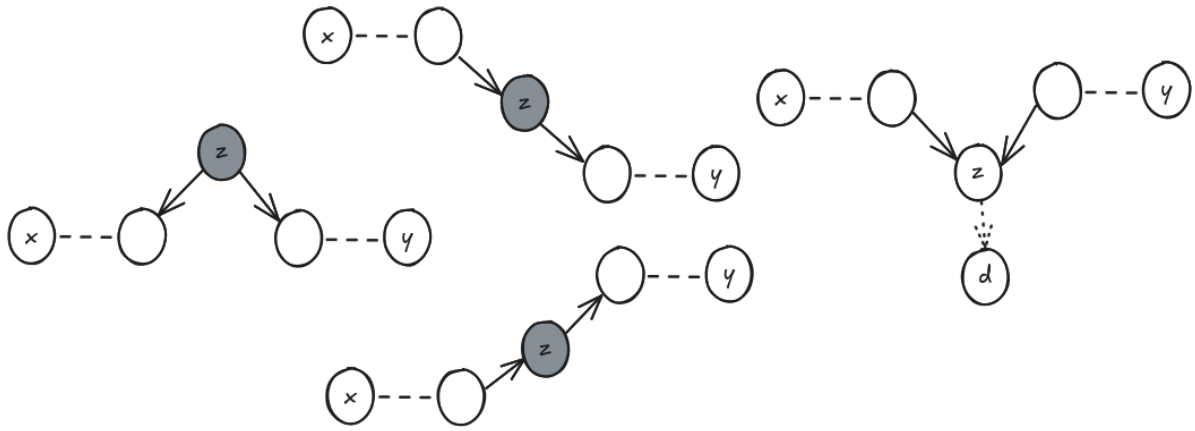


FIGURE 5.4 – Blocking configuration

If all such paths are blocked, then $\mathcal{X}$ and $\mathcal{Y}$ are de-separated by $\mathcal{Z}$.
If the variable sets $\mathcal{X}$ and $\mathcal{Y}$ are d-separated by $\mathcal{Z}$, they are independent conditional on $\mathcal{Z}$ in all probability distributions such a graph can represent.

It is important to notice that BN only makes independence statements not causal ones.

6 Bayesian networks and causality

In Bayesian network, we may have an imperfect definition of causality that states that variable A is causal for variable B if A and B are strongly dependent conditionally on all other random variables, and if A temporally occurs before B, unless B anticipates A.

Real-world phenomena can be modeled causally, and Bayesian networks naturally formalize causal problems through causal graphs. However, in general, BNs do not express direct causality relations ; they primarily represent dependencies between variables.

7 Learning from complete data : MLE of Bayesian network

In this section, we illustrates how to compute the MLE for CPT. We demonstrate the property that the MLE of a Bayesian network is the union of MLE applied to BN factors (i.e CPT) separately.
Let a BN whose variables are $\mathcal{X} = (X_1, \ldots, X_m)$, θ_i parameters of $p_{X_i|\text{parents}(X_i)}$, $\mathcal{D} = \{x_j\}_{1 \leq j \leq n}$ be a set of i.i.d samples with $x_j = (x_1 j, \ldots, x_m j)$.
To find the MLE of the parameters in a Bayesian network, we differentiate the log-likelihood function with respect to each parameter and set it to zero, leading to the following condition :

$$\frac{\partial \mathcal{L}(\mathcal{D}; \theta)}{\partial \theta} = 0 \Leftrightarrow \forall k, \frac{\partial \mathcal{L}(\mathcal{D}; \theta_k)}{\partial \theta_k} = 0 \tag{7.1}$$

$$p(\mathcal{D}|\theta) = \prod_{j=1}^{n} p(x_j|\theta) \tag{7.2}$$

$$= \prod_{j=1}^{n} \prod_{i=1}^{m} p_{X_i|\text{parents}(X_i),\theta_i}(x_{ij}) \tag{7.3}$$

Thus,

$$\log p(\mathcal{D}|\theta) = \sum_{j=1}^{n} \sum_{i=1}^{m} \log p_{X_i|\text{parents}(X_i),\theta_i}(x_{ij}) \tag{7.4}$$

We have $\forall k$,

$$\frac{\partial \log p(\mathcal{D}|\theta)}{\partial \theta_k} = \sum_{j=1}^{n} \sum_{i=1}^{m} \frac{\partial}{\partial \theta_k} \log p_{X_i|\text{parents}(X_i),\theta_i}(x_{ij}) \tag{7.5}$$

$$= \sum_{j=1}^{n} \frac{\partial}{\partial \theta_k} \log p_{X_k|\text{parents}(X_k),\theta_k}(x_{kj}) \tag{7.6}$$

$$= \frac{\partial}{\partial \theta_k} \left(\sum_{j=1}^{n} \log p_{X_k|\text{parents}(X_k),\theta_k}(x_{kj}) \right) \tag{7.7}$$

$$= \frac{\partial}{\partial \theta_k} \left(\sum_{j=1}^{n} \log \mathcal{L}(x_j; \theta_k) \right) \tag{7.8}$$

$$= \frac{\partial \log \mathcal{L}(\mathcal{D}; \theta_k)}{\partial \theta_k} \tag{7.9}$$

Then equation 7.1 finishes the demonstration.

8 Bayesian inference in Bayesian networks

Here we demonstrate that if priors of BN factors are independent, posteriors of BN factors are also independent.

We suppose that the priors of the BN factors are independent :

$$p(\theta) = \prod_{i=1}^{m} p(\theta_i) \tag{8.1}$$

Then,

$$p(\theta|\mathcal{D}) \propto \mathcal{L}(\mathcal{D};\theta) \tag{8.2}$$

$$\propto \left(\prod_{j=1}^{n}\prod_{i=1}^{m} p_{X_i|\text{parents}(X_i),\theta_i}(x_{ij})\right) \times \left(\prod_{i=1}^{m} p(\theta_i)\right) \tag{8.3}$$

$$\propto \prod_{i=1}^{m}\left(\left(\prod_{j=1}^{n} p_{X_i|\text{parents}(X_i),\theta_i}(x_{ij})\right) \times p(\theta_i)\right) \tag{8.4}$$

$$\propto \prod_{i=1}^{m} \mathcal{L}(\mathcal{D};\theta_i) \times p(\theta_i) = \prod_{i=1}^{m} p(\theta_i|\mathcal{D}) \tag{8.5}$$

Hence, Bayesian inference can be applied to variables one after the other.

9 Naive Bayes

Naive Bayes is generative approach which assumes the features X_i are conditionally independent given the class label :

$$\forall i, \forall j, X_i \perp X_j | Y \tag{9.1}$$

We illustrate Naive Bayes graph model in figure 9.1.

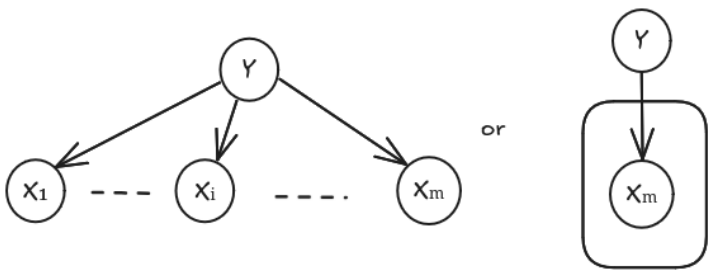


FIGURE 9.1 – Naive Bayes graph model : $k - 1 + 2km$ parameters

This allows us to write the class conditional density as a product of one dimensional densities :

$$p(X, Y = c) = p(y = c) \prod_{i=1}^{m} p(x_i|y = c) \tag{9.2}$$

With parameters θ, we have :

$$p(X, Y = c, \theta) = p(y = c, \theta) \prod_{i=1}^{m} p(x_i|y = c, \theta_{ic}) \tag{9.3}$$

The model is called **naive** since we do not expect the features to be independent, even conditional on the class label. However, even if the naive Bayes assumption is not true, it often results in classifiers that work well. One reason for this is that the model is quite simple (it only has $O(C \times m)$ parameters, for C classes and m features), and hence it is relatively immune to overfitting.

We compute the predicition of Y for a novel attribute X^* as follows :

$$p(Y|X^*,\theta) = \frac{p(X^*|Y,\theta)}{p(X^*|\theta)} p(Y|\theta) \propto \left(\prod_{i=1}^{m} p(X_i^*|Y,\theta)\right) \times p(Y|\theta) \tag{9.4}$$

This leads to :

$$p(Y=c|X^*,\theta) \propto \left(\prod_{i=1}^{m} p(X_i^*|Y=c,\theta)\right) \times p(Y=c|\theta) \tag{9.5}$$

- In the case of binary features, $x_j \in \{0,1\}$, we can use the Bernoulli distribution : $p(X|Y=c,\theta) = \prod_{j=1}^{m} \mathrm{Ber}(x_j|\theta_{jc})$, where θ_{jc} is the probability that feature j occurs in class c. This is sometimes called the **multivariate Bernoulli naive Bayes** model.
- In the case of categorical features, we can model use the multinoulli distribution : $p(X|Y=c,\theta) = \prod_{j=1}^{m} \mathrm{Cat}(X_j|\theta_{jc})$, where θ_{jc} is histogram over the m possible values for X_j in class c.
- In the case of real-valued features, we can use the Gaussian distribution : $p(X|Y=c,\theta) = \prod_{j=1}^{m} \mathcal{N}(X_j|\theta_{jc},\sigma_{jc}^2)$, where θ_{jc} is the mean of feature j when the class label is c, and σ_{jc}^2 is its variance.

9.1 Model fitting

We now discuss how to train a naive Bayes classifier. This usually means computing the MLE or the MAP estimate for the parameters.
The probability for a single data case is given by :

$$p(x_i,y_i|\theta) = p(y_i|\pi)\prod_{j=1}^{m} p(x_{ij}|\theta_j) = \prod_{c=1}^{C} \pi_c^{\mathbb{1}(y_i=c)} \prod_{j=1}^{m}\prod_{c=1}^{C} p(x_{ij}|\theta_{jc})^{\mathbb{1}(y_i=c)} \tag{9.6}$$

Hence, the log-likelihood is given by

$$\log p(\mathcal{D}|\theta) = \sum_{c=1}^{C}\sum_{i=1}^{N} \mathbb{1}(y_i=c)\log\pi_c + \sum_{j=1}^{m}\sum_{c=1}^{C}\sum_{i:y_i=c} \log p(x_{ij}|\theta_{jc}) \tag{9.7}$$

$$= \sum_{c=1}^{C} N_c \log\pi_c + \sum_{j=1}^{m}\sum_{c=1}^{C}\sum_{i:y_i=c} \log p(x_{ij}|\theta_{jc}) \tag{9.8}$$

where $N_c = \sum_{i=1}^{N} \mathbb{1}(y_i=c)$ is the number of examples in class c.
We see that this expression decomposes into a series of terms, one concerning π, and mC terms containing the θ_{jc}'s. Hence, we can optimize all these parameters separately.
The MLE for the class prior is given by :

$$\hat{\pi}_c = \hat{p}(Y=c) = \frac{N_c}{N} \tag{9.9}$$

The MLE for the likelihood depends on the type of distribution we choose to use for each feature.

- In the case of discrete features, we use categorical distribution and the MLE becomes :
$$\hat{\theta}_{jc}(v) = p(X_j = v|Y = c) = \frac{N_{jc}(v)}{N_c} \tag{9.10}$$
where $N_{jc}(v) = \sum_{i=1}^{N} \mathbb{1}(y_i = c)\mathbb{1}(x_{ij} = v)$
- In the case of binary features, the categorical distribution becomes the Bernoulli and the MLE becomes :
$$\hat{\theta}_{jc} = p(X_j = 1|Y = c) = \frac{N_{jc}}{N_c} \tag{9.11}$$
which is the empirical fraction of times that feature j is on in examples of class c ; where $N_{jc} = \sum_{i=1}^{N} \mathbb{1}(y_i = c)\mathbb{1}(x_{ij} = 1)$
- In the case of real-valued features, we can use a Gaussian distribution and the MLE is :
$$\hat{\theta}_{jc} = \frac{1}{N_c} \sum_{i:y_i=c} x_{ij} \tag{9.12}$$
$$\hat{\sigma}^2_{jc} = \frac{1}{N_c} \sum_{i:y_i=c} (x_{ij} - \hat{\theta}_{jc})^2 \tag{9.13}$$

It is extremely simple to implement this model fitting procedure. The Naive Bayes algorithm takes $O(nm)$ time. The method is easily generalized to handle features of mixed type. This simplicity is one reason the method is so widely used.
MAP and Bayesian inference are not typically useful for Naive Bayes in most cases because Naive Bayes already makes strong independence assumptions between features, which simplifies the model and often leads to sufficiently accurate predictions with just the Maximum Likelihood Estimation (MLE) of parameters. In Naive Bayes, we assume that the features are conditionally independent given the class, and thus, the likelihood of the data can be computed independently for each feature. Adding a prior distribution (as in MAP or full Bayesian inference) does not often improve the model's performance, because the parameter estimation is already fairly straightforward with MLE and does not require incorporating complex priors or posterior updates. Furthermore, in many practical applications, the computational cost of performing full Bayesian inference is not justified, especially when the data is large and the model's simplicity through MLE is already effective.

Example 9.1 (Spam Detection using Naive Bayes)**.** We are given a set of email messages and we aim to classify whether a message is spam or not using the Naive Bayes classifier. The emails are represented using the **Bag of Words** model, where the presence (or absence) of certain words is encoded as binary features. In this case, we use the words `earn`, `million`, `account`, and `password` to model the emails.
Dataset : Let X_1 to X_4 represent the presence or absence of the words `earn`, `million`, `account`, and `password`, respectively. We are given 10 labeled examples as shown below :

earn	million	account	password	Class
1	1	0	0	Spam
0	0	1	1	Spam
0	1	1	0	Not Spam
1	1	0	0	Spam
0	0	0	0	Not Spam
1	0	0	0	Spam
1	0	0	0	Not Spam
0	0	0	1	Spam
1	0	1	1	Spam
0	1	1	1	Not Spam

The prior probabilities of spam and not spam can be computed using equation 9.9 :
$P(\text{Spam}) = \frac{6}{10} = 0.6$
$P(\text{Not Spam}) = \frac{4}{10} = 0.4$
For each word, we compute the conditional probability of its occurrence given the class label using equation 9.10.
For the word earn :

$$P(\text{earn} = 1|\text{Spam}) = \frac{4}{6}, \quad P(\text{earn} = 1|\text{Not Spam}) = \frac{1}{4}$$

For the word million :

$$P(\text{million} = 1|\text{Spam}) = \frac{2}{6}, \quad P(\text{million} = 1|\text{Not Spam}) = \frac{2}{4}$$

For the word account :

$$P(\text{account} = 1|\text{Spam}) = \frac{2}{6}, \quad P(\text{account} = 1|\text{Not Spam}) = \frac{2}{4}$$

For the word password :

$$P(\text{password} = 1|\text{Spam}) = \frac{3}{6}, \quad P(\text{password} = 1|\text{Not Spam}) = \frac{1}{4}$$

Given a new message with the words `earn` and `million`, we calculate the posterior probabilities for Spam and Not Spam using Bayes' theorem.
Using Bayes' rule, we compute :

$$P(Y = \text{Spam}|\text{earn, million}) \propto P(Y = \text{Spam}) \times P(\text{earn} = 1|\text{Spam}) \times P(\text{million} = 1|\text{Spam}) \tag{9.14}$$

$$\times P(\text{account} = 0|\text{Spam}) \times P(\text{password} = 0|\text{Spam}) \tag{9.15}$$

$$\propto \frac{6}{10}\frac{4}{6}\frac{2}{6}\left(1 - \frac{2}{6}\right)\left(1 - \frac{3}{6}\right) = \frac{2}{45} \tag{9.16}$$

Similarly, for Not Spam :

$$P(Y = \text{Not Spam}|\text{earn, million}) \propto P(Y = \text{Not Spam}) \times P(\text{earn} = 1|\text{Not Spam}) \times P(\text{million} = 1|\text{Not S} \tag{9.17}$$

$$\times P(\text{account} = 0|\text{Not Spam}) \times P(\text{password} = 0|\text{Not Spam}) \tag{9.18}$$

$$\propto \frac{4}{10}\frac{1}{4}\frac{2}{4}\left(1 - \frac{2}{4}\right)\left(1 - \frac{1}{4}\right) = \frac{3}{160} \tag{9.19}$$

Now, we normalize the probabilities :
$P(Y = \text{Spam}|\text{earn, million}) = \frac{\frac{2}{45}}{\frac{2}{45}+\frac{3}{160}} = 0.7$
$P(Y = \text{Not Spam}|\text{earn, million}) = \frac{\frac{3}{160}}{\frac{2}{45}+\frac{3}{160}} = 0.3$
Thus, the message "earn+million" is classified as **Spam**.

Example 9.2 (Extension of Naive Bayes to continuous features)**.** TODO

9.2 Bayesian Naive Bayes

The trouble with MLE is that it can lead to overfitting. For example, consider the scenario where a feature always appears in both classes. Consequently, we estimate : $\hat{\theta}_{ic} = 1$.
Now, what happens if we encounter a new email that does not contain this word ? Our algorithm will fail, as we will compute : $p(Y = c|x, \hat{\theta}) = 0$ for both classes !
A simple remedy to overfitting is to use a Bayesian approach.

$$p(\theta) = p(\pi) \prod_{i=1}^{m} \prod_{c=1}^{C} p(\theta_{ic}). \tag{9.20}$$

We choose a Dirichlet prior $\text{Dir}(\alpha)$ for π and a Beta prior $\text{Beta}(\beta_0, \beta_1)$ for each θ_{jc}. We may also choose Gaussian prior on mean for continuous fearures with fixed variance. Often, we set : $\alpha = 1, \quad \beta_0 = 1, \quad \beta_1 = 1$.
We obtain the following factored posterior :

$$p(\theta|\mathcal{D}) = p(\pi|\mathcal{D}) \prod_{i=1}^{m} \prod_{c=1}^{C} p(\theta_{ic}|\mathcal{D}). \tag{9.21}$$

The posteriors take the form :

$$p(\pi|\mathcal{D}) = \text{Dir}(N_1 + \alpha_1, \ldots, N_C + \alpha_C), \tag{9.22}$$

$$p(\theta_{ic}|\mathcal{D}) = \text{Beta}((N_c - N_{ic}) + \beta_0, N_{ic} + \beta_1). \tag{9.23}$$

Thus, to compute the posterior, we simply update the prior counts with the empirical counts from the likelihood.

Naive Bayes is a simple generative model for classification, very flexible and general (no restriction on type of features). However, it does not capture feature dependencies within classes. Consequently, accuracy is limited for non-trivial problems and output distributions $p(X|y)$ are not calibrated (tend to be degenerated).

10 Calibration of output distributions

TODO

11 Classification-related notions

11.1 False positive vs false negative tradeoff

A **false positive** (or **false alarm**), arises when we estimate $\hat{y} = 1$ but the truth is $y = 0$; a **false negative** (or **missed detection**) arises when we estimate $\hat{y} = 0$ but the truth is $y = 1$. The $0 - 1$ loss treats these two kinds of errors equivalently. However, we can consider the following more general loss matrix :

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	0	L_{FN}
$y = 0$	L_{FP}	0

TABLE 11.1 – Confusion matrix with loss terms

where L_{FN} is the cost of a false negative, L_{FP} is the cost of a false positive. The posterior expected loss for the two possible actions is given by :

$$\rho(\hat{y} = 0|x) = L_{FN} p(y = 1|x) \tag{11.1}$$

$$\rho(\hat{y} = 1|x) = L_{FP} p(y = 0|x) \tag{11.2}$$

Hence, we should pick class $\hat{y} = 1$ iff :

$$\rho(\hat{y} = 0|x) > \rho(\hat{y} = 1|x) \quad \Leftrightarrow \quad \frac{p(y = 1|x)}{p(y = 0|x)} > \frac{L_{FP}}{L_{FN}} \tag{11.3}$$

If $L_{FN} = cL_{FP}$, it is easy to show that we should pick $\hat{y} = 1$ iff :

$$\frac{p(y = 1|x)}{p(y = 0|x)} > \tau, \tag{11.4}$$

where $\tau = \frac{c}{1+c}$. For example, if a false negative costs twice as much as a false positive, so $c = 2$, then we use a decision threshold of $\frac{2}{3}$ before declaring a positive.

	Truth		
Estimate	1	0	Σ
1	TP	FP	$\hat{N}_+ = TP + FP$
0	FN	TN	$\hat{N}_- = FN + TN$
Σ	$N_+ = TP + FN$	$N_- = FP + TN$	$N = TP + FP + FN + TN$

TABLE 11.2 – N_+ is the true number of positives, $\hat{N}_+$ is the called number of positives, N_- is the true number of negatives, $\hat{N}_-$ is the called number of negatives.

	$y = 1$	$y = 0$
$\hat{y} = 1$	$TP/N_+ = \text{TPR} = \text{sensitivity} = \text{recall}$	$FP/N_- = \text{FPR} = \text{type I}$
$\hat{y} = 0$	$FN/N_+ = \text{FNR} = \text{miss rate} = \text{type II}$	$TN/N_- = \text{TNR} = \text{specificity}$

TABLE 11.3 – Estimating $p(\hat{y}|y)$ from a confusion matrix. Abbreviations : FNR = false negative rate, FPR = false positive rate, TNR = true negative rate, TPR = true positive rate.

Consider a decision rule of the form $\mathbb{1}(f(x) > \tau)$, where $f(x)$ represents a confidence measure that $y = 1$. This function should be monotonically related to $p(y = 1|x)$ but does not necessarily need to be a probability. The parameter τ acts as a threshold, determining classification outcomes. For each choice of τ, we compute true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), which together form the **confusion matrix** illustrated in table 11.2.
From this matrix, we derive key performance metrics illustrated in table 11.3 :

- **True Positive Rate (TPR)** : Also known as recall or sensitivity, computed as

$$TPR = \frac{TP}{N_+} \approx p(\hat{y} = 1|y = 1) \tag{11.5}$$

- **False Positive Rate (FPR)** : Also called the false alarm rate or Type I error, given by

$$FPR = \frac{FP}{N_-} \approx p(\hat{y} = 1|y = 0) \tag{11.6}$$

These values vary depending on the threshold τ. Instead of selecting a fixed τ, we can analyze system performance across different thresholds by plotting **TPR vs. FPR**, forming the **Receiver Operating Characteristic (ROC) curve)**.
A **perfect classifier** achieves the upper-left point $(0, 1)$, where $FPR = 0$ and $TPR = 1$, meaning no false positives and complete detection of true positives. In contrast, a classifier making random guesses would follow the diagonal $TPR = FPR$. An ideal classifier's ROC curve *hugs* the left and top borders, showing superior discrimination.
A common way to quantify ROC performance is the **Area Under the Curve (AUC)**, where higher values indicate better classification (maximum value is 1). Another key metric is the **Equal Error Rate (EER)**, defined as the threshold where $FPR = FNR$ (false negative rate). The EER can be determined graphically as the intersection of the ROC curve with the diagonal from $(1, 0)$ to $(0, 1)$. Lower EER values indicate a better-performing system.

The formula of biased accuracy is :

$$\text{accuracy} = \frac{TP + TN}{N} = \frac{N_+}{N_+ + N_-}\text{sensitivity} + \frac{N_-}{N_+ + N_-}\text{specifity} \tag{11.7}$$

The unbiased accuracy is :

$$\text{accuracy} = \frac{1}{2}\text{sensitivity} + \frac{1}{2}\text{specifity} \tag{11.8}$$

11.2 Precision recall curves

When detecting rare events for example the number of negative instances may be typically very large. In such cases, comparing the true positive rate (TPR), given by $TPR = TP/N_+$, to the false positive rate (FPR), given by $FPR = FP/N_-$, may not be particularly informative. Since the FPR will be very small, most of the meaningful variations in the ROC curve will occur on its extreme left. To address this, an alternative approach is to plot the TPR against the absolute number of false positives rather than the false positive rate.
In some scenarios, the concept of a "negative" instance is not well-defined. It is preferable to use evaluation metrics that focus only on the positive instances. One such measure is **precision**, defined as $TP/\hat{N}_+ = p(y = 1|\hat{y} = 1)$, which quantifies the proportion of detected instances that are truly positive. Another measure is **recall**, defined as $TP/N_+ = p(\hat{y} = 1|y = 1)$, which represents the proportion of actual positives that have been correctly detected. Precision evaluates the reliability of detections, while recall assesses the completeness of the detection process. Given predicted labels $\hat{y}_i \in \{0, 1\}$ and true labels $y_i \in \{0, 1\}$, these metrics can be estimated as follows :

$$P = \frac{\sum_i y_i \hat{y}_i}{\sum_i \hat{y}_i}, \quad R = \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i} \tag{11.9}$$

A **precision-recall curve** is a plot of precision against recall as the decision threshold τ varies.

11.3 F-score

For a fixed threshold, one can compute a single precision and recall value. These are often combined into a single statistic called the **F-score**, or more specifically the **F1-score**, which is the harmonic mean of precision and recall. It is useful if you need a simple way to compare two classifiers. The harmonic mean gives much more weight to low values. As a result, the classifier will only get a high F1-score if both recall and precision are high. The F1-score favors classifiers that have similar precision and recall.

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2PR}{P + R} \tag{11.10}$$

Using equation 11.9, this can be rewritten as :

$$F_1 = \frac{2 \sum_{i=1}^{N} y_i \hat{y}_i}{\sum_{i=1}^{N} y_i + \sum_{i=1}^{N} \hat{y}_i} \tag{11.11}$$

12 References

- Murphy, K. P. (2012). *Machine Learning : a Probabilistic Perspective*. MIT Press. `https://probml.github.io/pml-book/book0.html`
- Murphy, K. P. (2023). *Probabilistic Machine Learning : Advanced Topics*. MIT Press. `http://probml.github.io/book2`
- Statistical Models - SDI Metz